

Can LLMs Implement Architectural Tactics? Early Results

Saveliy Chertkov
Innopolis University
Innopolis, Russia
s.chertkov@innopolis.university

Andrey Sadovykh
Innopolis University
Innopolis, Russia
a.sadovykh@innopolis.ru

Abstract

Software maintainability is a critical quality attribute that directly affects system evolution cost and reliability. Improving maintainability often requires architectural-level modifications demanding specialized expertise and significant manual effort. While Large Language Models (LLMs) have shown strong capabilities in code understanding and generation, their potential for systematic architecture-level software quality improvement remains largely unexplored.

This paper presents an automated pipeline that leverages an LLM to improve software maintainability through architectural tactic implementation. The pipeline operates in three stages: (1) architecture detection, where the LLM analyzes repository structure and identifies the dominant architectural pattern; (2) architectural tactic selection, where the LLM chooses an appropriate tactic from a predefined catalog based on the detected architecture and expected maintainability impact; and (3) tactic implementation using a Test-Driven Development approach to ensure correctness. The pipeline follows a Pipes and Filters architecture and operates without manual intervention.

We applied the pipeline to 158 open-source Python backend repositories collected and filtered from GitHub. Preliminary results show that the LLM successfully detected architectural patterns (e.g., modular monolith, layered, script-based) in 106 repositories and selected contextually appropriate tactics for 77 repositories, with Reduced Coupling being the most frequently chosen tactic. Baseline maintainability was measured using the Radon static analysis tool, with the Maintainability Index as the primary metric.

This work contributes a reproducible methodology for LLM-driven architectural improvement and provides early empirical evidence on the feasibility of using LLMs for architecture-level analysis and tactic reasoning.

ACM Reference Format:

Saveliy Chertkov and Andrey Sadovykh. 2026. Can LLMs Implement Architectural Tactics? Early Results. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern software engineering faces a persistent maintenance challenge, with studies indicating that 60–80% of total life-cycle costs are dedicated to post-deployment modifications [1]. While code-level refactoring is well-established, there is a significant gap in the automated application of higher-level design decisions. This review examines the intersection of architectural tactics, quality attribute models, and the emerging capabilities of Large Language Models (LLMs) in automating structural transformations.

2 Architectural Tactics and Quality Attributes

Software architecture is fundamentally defined by its elements, form, and rationale [8], or alternatively, as a collection of components and connectors in a specific configuration [2]. Within these frameworks, architectural tactics serve as fine-grained design decisions intended to achieve specific quality attribute responses [1].

The ISO/IEC 25010 standard provides a comprehensive model for evaluating these attributes, specifically decomposing maintainability into modularity, analysability, modifiability, reusability, and testability [1]. Research by Rahmati suggests that while architectural patterns (e.g., MVC, Layered) provide a structural foundation, their effectiveness is highly context-dependent; misapplication can lead to a 10% increase in maintenance effort [10]. Consequently, the selection and implementation of specific tactics, such as "Use an Intermediary" or "Split Module," are critical for targeted quality improvement [1].

3 The Role of LLMs in Software Refactoring

The "LLM Revolution" has introduced new paradigms for code transformation. Recent advancements in multi-agent frameworks, such as MANTRA, have demonstrated an 82.8% success rate in producing compilable, test-passing refactored code [12]. However, empirical analysis of AI coding agents reveals a significant "Scale Gap": approximately 96.3% of current agentic refactorings are confined to a single file [3].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EASE 2026, Glasgow, United Kingdom

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Furthermore, research into prompt engineering indicates that while LLMs can identify up to 86.7% of refactoring opportunities, their performance in complex tactic synthesis remains limited [5]. For instance, in security-related tactic implementation, models may achieve high syntactic correctness (95%) but fail significantly in semantic correctness (5%) due to an inability to coordinate inter-procedural changes across multiple classes [11].

4 Identified Research Gaps

The synthesis of current literature identifies a "Transformation Gap" between tactic detection and implementation:

- **Contextual Constraints:** Current LLM-based tools operate primarily at the method or class level, lacking the system-wide awareness required for architectural restructuring [3, 7].
- **Measurement Crisis:** There is a documented lack of agreement among static analysis tools (less than 0.4% agreement in some cases), complicating the validation of quality improvements [4].
- **Implementation Void:** While numerous methods exist for detecting the need for architectural tactics, there is a lack of automated, LLM-driven mechanisms to translate architectural intent into concrete, multi-file code changes [6, 11].

5 Methodology

5.1 Research Design

This study employs a quantitative pre-test/post-test experimental design to evaluate the efficacy of Large Language Models (LLMs) in structural code transformation. Following the principles of evidence-based software engineering [1], we define the independent variable as the application of LLM-based architectural tactics, while the dependent variable is software maintainability. Each repository in the dataset serves as its own control, allowing for a paired-sample comparison of metrics before and after the intervention to mitigate the impact of inherent project variance.

5.2 Experimental Pipeline

The systematic pipeline is structured into seven sequential phases: (1) Automated repository selection; (2) Baseline static analysis; (3) Architectural pattern detection; (4) Tactic selection based on detected deficiencies; (5) Iterative tactic implementation; (6) Post-intervention static analysis; (7) Comparative statistical analysis.

5.3 Dataset Preparation and Filtering

To ensure the relevance and quality of the analyzed software, repositories were sourced from public Python-based backend projects on GitHub. We implemented a multi-stage filtering process to exclude "toy" projects and focus on maintainable industrial-like codebases [10]. The inclusion criteria required:

- Presence of a valid `requirements.txt` for dependency analysis;
- A minimum threshold of Python source files to ensure architectural complexity;
- Exclusion of Machine Learning-heavy projects to avoid data-science-specific coding patterns (e.g., Jupyter Notebooks);
- Assessment of project maturity through metadata such as star counts and commit history.

5.4 Maintainability Measurement Framework

The evaluation of maintainability relies on the ISO/IEC 25010 standard, focusing specifically on modularity and analysability [1]. The primary quantitative metric is the Maintainability Index (MI), calculated via the Radon framework.

However, recognizing the "Agreement Crisis" in static analysis—where different tools often show less than 0.4% consensus on quality issues [4]—our methodology supplements MI with a multi-dimensional metric suite:

- **Structural Complexity:** Average fan-out and package coupling;
- **Documentation Quality:** Docstring coverage and README presence;
- **Delta Measurement:** $\Delta MI = MI_{after} - MI_{before}$, providing a normalized growth rate of code quality.

5.5 LLM-Driven Transformation Process

The implementation process leverages the reasoning capabilities of LLMs, structured into three distinct logical layers:

Architecture Detection: The model analyzes the directory structure and module relationships to infer the underlying architectural style (e.g., Layered, Hexagonal) [2, 8].

Tactic Selection: Based on identified "smells" or high coupling, the system selects an appropriate tactic (e.g., "Use an Intermediary") from the catalog established by Bass et al. [1].

Tactic Implementation: Currently, the implementation follows an *iterative incremental approach*, where the LLM performs small, verifiable changes to the codebase. While literature suggests that Test-Driven Development (TDD) significantly improves the reliability of LLM refactoring [9, 12], the integration of a full TDD cycle—including automated failing test generation and minimal implementation—is designated for **future iterative improvements** of the pipeline.

6 Implementation

The experimental framework is implemented as an automated Python-based orchestration engine. To ensure isolation and reproducibility, the system utilizes shallow clones of target repositories, which are processed sequentially and purged after data extraction.

The LLM integration utilizes structured prompt engineering, enforcing JSON-formatted outputs to facilitate automated

parsing of architectural decisions. This approach addresses the "Scale Gap" identified in recent studies, where 96.3% of AI-driven changes are typically limited to single files [9]. By maintaining a global architectural context in the prompt, our implementation enables the coordination of changes across multiple modules. All artifacts, including logs of LLM reasoning and intermediate code states, are preserved to ensure full traceability of the architectural evolution.

7 Results and Discussion

7.1 Overall Quantitative Impact on Maintainability

The experimental results across the analyzed repositories reveal a nuanced impact of LLM-based architectural interventions on the Maintainability Index (MI). As illustrated in the comparative distribution (Figure 1), a significant portion of the projects maintained their baseline stability or showed incremental improvements.

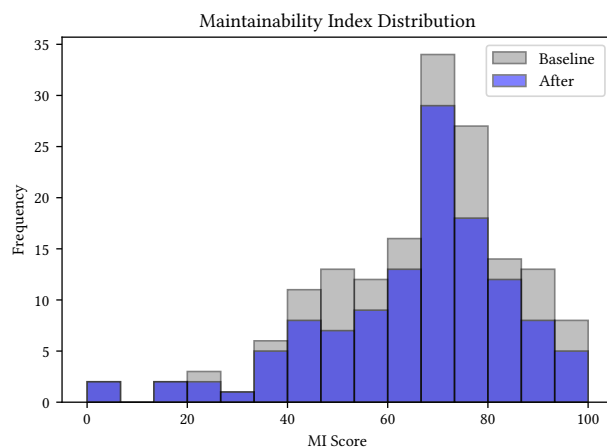


Figure 1: Distribution of Maintainability Index (MI) scores: Baseline vs. Post-Intervention.

The data highlights a "Stability Plateau": in approximately 58% of cases (e.g., *DrissionPage*, *Sublist3r*), the MI remained unchanged. This suggests that while the LLM successfully performed internal refactoring, the changes were either not extensive enough to shift the aggregate MI score, or the static analysis tool (Radon) did not perceive the specific structural changes as complexity reductions.

7.2 Analysis of Success Cases and Tactic Efficacy

The implementation of architectural tactics led to measurable success in several instances. As shown in the analysis of the most significant shifts (Figure 2), the highest improvement in MI was recorded in the *get-subscribe* repository.

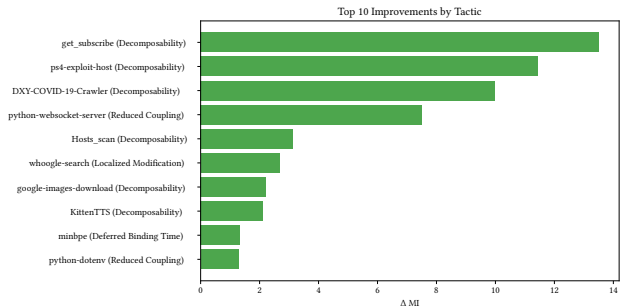


Figure 2: Top 10 Maintainability Index (MI) improvements categorized by applied tactic.

This success was achieved through the **Decomposability** tactic applied to a **script_based** architecture. By decomposing large, dense scripts into modular components, the LLM effectively reduced individual file complexity, which is highly rewarded by the MI formula. Similarly, in the *whoogle-search* repository, the MI increased from 67.06 to 69.74 after applying **Localized Modification**, representing a clear enhancement in code health within a layered system.

7.3 System Robustness and Failure Rates

A critical finding of this study is the high risk associated with automated architectural shifts without human oversight. While many projects remained stable, Figure 3 demonstrates that in 23.1% of cases, the intervention resulted in a "Regression (Null)" status.

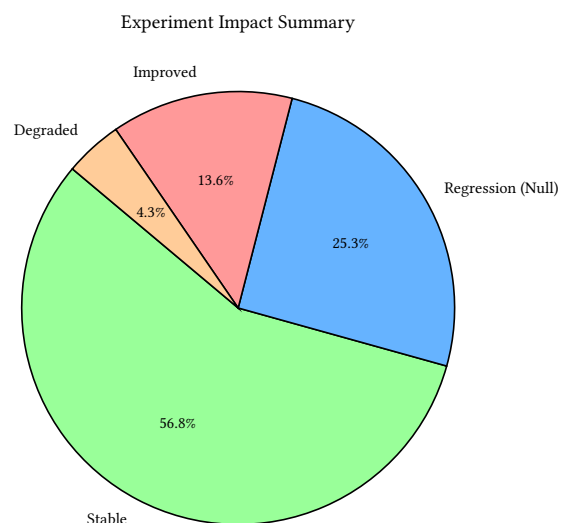


Figure 3: Summary of intervention impact: percentage of improved, stable, and failed (null) projects.

These cases represent instances where the LLM introduced syntax errors, broken dependencies, or architectural changes that made the project impossible to launch. This "fragility" of automated refactoring underscores the necessity of the proposed TDD-based verification loop to ensure that tactical improvements do not compromise fundamental system integrity.

7.4 Architectural Influence on LLM Performance

The distribution of architectures in the dataset (Figure 4) shows a predominance of **modular monoliths** systems.

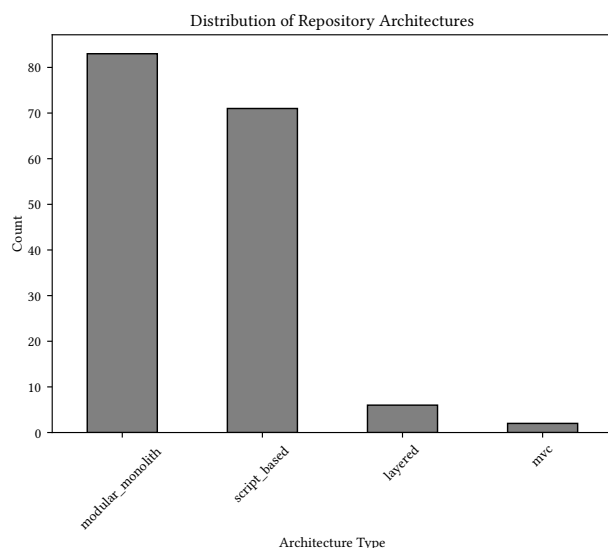


Figure 4: Distribution of baseline architectures across the analyzed repositories.

The results suggest that "Modular Monoliths" and "Layered" systems provided a more predictable context for the LLM, leading to more consistent refactoring. In contrast, "Script-based" projects, while showing the highest potential for gain (as seen in *get-subscribe*), also lacked the structural boundaries required for complex tactics like "Deferred Binding Time". This aligns with the "Context Blindness" identified in recent studies [3], where LLMs struggle to manage large-scale architectural shifts in unstructured codebases without explicit modular cues.

8 Conclusion

This study investigated the potential of Large Language Models to automate the implementation of architectural tactics for software maintainability. Our findings lead to several key conclusions:

First, LLMs are capable of identifying and implementing architectural tactics that result in measurable improvements in code maintainability, particularly in well-structured layered systems. The successful improvement of the Maintainability

Index in several repositories demonstrates that AI can move beyond simple code-level refactoring to address higher-level design concerns.

Second, the "Transformation Gap" remains a significant hurdle. The prevalence of neutral results (no change in MI) and occasional regressions highlight that current LLM pipelines lack the sophisticated architectural reasoning required to consistently improve complex systems. The reliance on a single metric (MI) may also fail to capture the qualitative benefits of better separation of concerns.

Finally, while the current iterative incremental approach is promising, it is limited by the absence of a robust verification loop. Future work should focus on integrating the proposed Test-Driven Development (TDD) framework to ensure that architectural changes preserve behavioral semantics while improving structure. Moving forward, the development of architecture-aware agents that can maintain global context across multiple files will be essential for bridging the gap between architectural theory and automated practice.

References

- [1] Len Bass, Paul Clements, and Rick Kazman. 2021. *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- [2] David Garlan and Mary Shaw. 1993. An introduction to software architecture. 1, 3 (1993).
- [3] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. In *arXiv preprint arXiv:2511.04824*. arXiv:2511.04824
- [4] Valentina Lenarduzzi, Fabiano Pecorelli, Nyti Saarimaki, Savanna Lujan, and Fabio Palomba. 2023. A critical comparison on six static analysis tools: Detection, agreement, and precision. 198 (2023), 111575.
- [5] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2025. Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study. *Automated Software Engineering* 32, 1 (2025). doi:10.1007/s10515-025-00500-0
- [6] Gastón Márquez, Hernán Astudillo, and Rick Kazman. 2022. Architectural tactics in software architecture: A systematic mapping study. *Journal of Systems and Software* 197 (2022), 111558. doi:10.1016/j.jss.2022.111558
- [7] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah AlOmar. 2025. Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software* (2025), 112762. doi:10.1016/j.jss.2025.112762
- [8] Dewayne E. Perry and Alexander L. Wolf. 1992. Foundations for the study of software architecture. 17, 4 (1992), 40–52. doi:10.1145/141874.141884
- [9] Yonnel Chen Kuang Piao, Jean Carlos Paul, L. Da Silva, Arghavan Moradi Dakhel, Mohammad Hamdaqa, and Foutse Khomh. 2025. Refactoring with LLMs: Bridging Human Expertise and Machine Understanding. In *arXiv preprint arXiv:2510.03914*. arXiv:2510.03914
- [10] Zahed Rahmati and Mohammad Tanhaei. 2021. Ensuring software maintainability at software architecture level using architectural patterns. 2, 1 (2021), 81–102. doi:10.22060/ajmc.2021.19232.1044
- [11] Seyyed Ehsan Shokri and Raffi Khatchadourian. 2024. IPSynth: Interprocedural Program Synthesis for Software Architecture Recovery and Architectural Tactics Detection. In *arXiv preprint arXiv:2403.10836*. arXiv:2403.10836
- [12] Yisen Xu, Feng Lin, Jinqiu Yang, Tse-Hsun Chen, and Nikolaos Tsantalis. 2025. MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration. In *arXiv preprint arXiv:2503.14340*. arXiv:2503.14340